

KARPAGAVINAYAGA
COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ROBOTICS AND AUTOMATION

REGULATIONS 2021



**OCS351–ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING FUNDAMENTALS LAB**

Name:

Reg.No:

Department:

Year/Sem:

TABLE OF CONTENT					
Exp.No.	Date	Title	Page No.	Sign.	Remarks
1.		Implement breadth first search			
2.		Implement depth first search			
3.		Analysis of breadth first and depth first search in terms of time and space			
4.		Implement and compare Greedy and A* algorithms.			
5.		Implement the non-parametric locally weighted regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs			
6.		Write a program to demonstrate the working of the decision tree based algorithm.			
7.		Build an artificial neural network by implementing the back propagation algorithm and test the same using appropriate data sets.			
8.		Write a program to implement the naïve Bayesian classifier.			
9.		Implementing neural network using self-organizing maps			
10.		Implementing k-Means algorithm to cluster a set of data.			
11.		Implementing hierarchical clustering algorithm.			
12.		Installation of gnu-prolog and Study of Prolog (gnu-prolog) – Case Study			

BREADTH FIRST SEARCH

EXP NO: 1

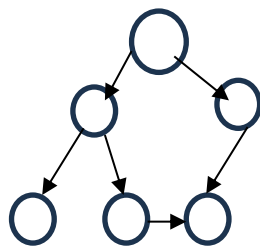
DATE:

Aim:

To write a python program to implement breadth first search.

Algorithm:

1. Start from the source node.
2. Mark the source node as visited and add it to a queue.
3. While the queue is not empty:
 - Remove the front node from the queue.
 - Visit the node.
 - Add all unvisited neighboring nodes to the queue and mark them as visited.
4. Repeat until the queue becomes empty.
5. Stop the program.



Program:

```
from collections import deque
```

```
# Graph represented as an adjacency list
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}
```

```
def bfs(graph, start):  
    visited = set()  
    queue = deque([start])
```

```
    visited.add(start)
```

```
    while queue:  
        node = queue.popleft()  
        print(node, end=" ")
```

```
        for neighbor in graph[node]:
```

```
        if neighbor not in visited:  
visited.add(neighbor)  
queue.append(neighbor)
```

```
# Starting BFS from node A  
print("BFS Traversal:")  
bfs(graph, 'A')
```

Output:

```
BFS Traversal:  
A B C D E F
```

Result:

Thus, the given python program to implement breadth first search is executed and verified successfully.

DEPTH FIRST SEARCH

EXP NO: 2

DATE:

Aim:

Write a program to implement depth first search.

Algorithm:

Step 1: Start.

Step 2: Create a graph using an adjacency list.

Step 3: Create an empty set visited to keep track of visited nodes.

Step 4: Define a function dfs(node).

Step 5: Check whether the current node is already in visited.

- If not visited:
 - Print the node.
 - Add the node to visited.

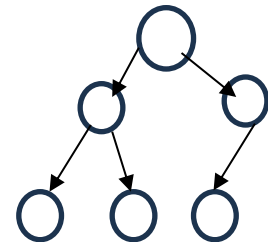
Step 6: Traverse all neighboring nodes of the current node.

- For each neighbor:
 - Call dfs(neighbor) recursively.

Step 7: Start the DFS traversal by calling dfs('A').

Step 8: Continue until all reachable nodes have been visited.

Step 9: Stop.



Program:

Graph represented as an adjacency list

```
graph = {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['D', 'E'],
```

```
    'C': ['F'],
```

```
    'D': [],
```

```
    'E': [],
```

```
    'F': []
```

```
}
```

```
visited = set()
```

```
def dfs(node):
```

```
    if node not in visited:
```

```
        print(node, end=" ")
```

```
        visited.add(node)
```

```
        for neighbor in graph[node]:  
            dfs(neighbor)  
  
# Start DFS from node A  
print("DFS Traversal starting from A:")  
dfs('A')
```

Output:

DFS Traversal starting from A:
A B D E C F

Result:

Thus, the given python program to implement depth first search is executed and verified successfully.

ANALYSIS OF BREADTH FIRST SEARCH (BFS) AND DEPTH FIRST SEARCH (DFS) IN TERMS TIME AND SPACE COMPLEXITY

EXP NO: 3

DATE:

Aim:

To analyze the performance of Breadth First Search (BFS) and Depth First Search (DFS) algorithms in terms of time complexity and space complexity using Python.

Theory

A graph traversal algorithm is used to visit all vertices of a graph systematically.

Breadth First Search (BFS)

- BFS traverses a graph level by level.
- It uses a Queue (FIFO) data structure.
- BFS is useful for finding the shortest path in an unweighted graph.

Depth First Search (DFS)

- DFS traverses a graph by exploring a branch as deeply as possible before backtracking.
- It uses Recursion or a Stack (LIFO).
- DFS is useful for cycle detection, topological sorting, and backtracking problems.

Algorithm:

Algorithm for BFS

1. Start from the source vertex.
2. Mark the source vertex as visited.
3. Insert the source vertex into a queue.
4. While the queue is not empty:
 - Remove a vertex from the queue.
 - Visit the vertex.
 - Add all unvisited adjacent vertices to the queue.
5. Repeat until the queue becomes empty.

Algorithm for DFS

1. Start from the source vertex.
2. Mark the source vertex as visited.
3. Visit the vertex.
4. Recursively visit all unvisited adjacent vertices.
5. Continue until all reachable vertices are visited.

Programs:

```
from collections import defaultdict, deque
```

```
class Graph:
```

```
    def __init__(self):
```

```
self.graph = defaultdict(list)
```

```
    def addEdge(self, u, v):
```

```
self.graph[u].append(v)
```

```
    # Breadth First Search
```

```
    def BFS(self, start):
```

```
        visited = set()
```

```
        queue = deque()
```

```
visited.add(start)
```

```
queue.append(start)
```

```
print("BFS Traversal:", end=" ")
```

```
    while queue:
```

```
        vertex = queue.popleft()
```

```
print(vertex, end=" ")
```

```
        for neighbour in self.graph[vertex]:
```

```
            if neighbour not in visited:
```

```
visited.add(neighbour)
```

```
queue.append(neighbour)
```

```
print()
```

```
    # Depth First Search
```

```
    def DFS(self, start):
```

```
        visited = set()
```

```
print("DFS Traversal:", end=" ")
```

```
self.DFS_util(start, visited)
```

```
print()
```

```
    def DFS_util(self, vertex, visited):
```

```

visited.add(vertex)

print(vertex, end=" ")

    for neighbour in self.graph[vertex]:

        if neighbour not in visited:

self.DFS_util(neighbour, visited)

# Driver Code

g = Graph()

g.addEdge(0, 1)

g.addEdge(0, 2)

g.addEdge(1, 3)

g.addEdge(1, 4)

g.addEdge(2, 5)

g.addEdge(2, 6)

print("Graph Traversal Starting from Vertex 0\n")

g.BFS(0)

g.DFS(0)

```

Output:

Graph Traversal Starting from Vertex 0

BFS Traversal: 0 1 2 3 4 5 6

DFS Traversal : 0 1 3 4 2 5 6

Result:

Thus, the given python program has been analysed the BFS and DFS in terms of time and space complexity.

IMPLEMENTATION AND COMPARISON OF GREEDY BEST-FIRST

SEARCH AND A* SEARCH ALGORITHMS

EXP NO: 4

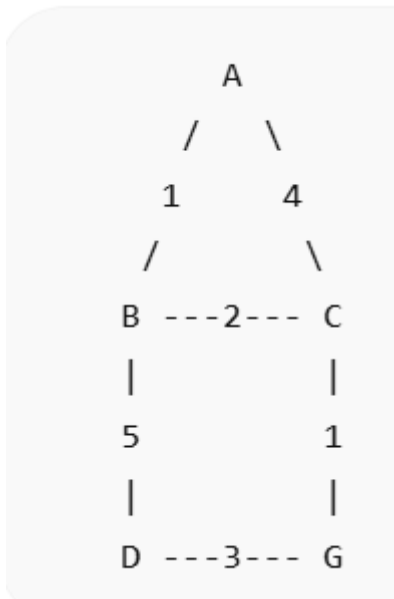
DATE:

Aim :

To implement and compare Greedy Best-First Search and A* Search algorithms using Python.

Problem Statement

Find a path from A (Start Node) to G (Goal Node).



Heuristic Values

Node	$h(n)$
A	7
B	6
C	2
D	3
G	0

Algorithm

Greedy Best-First Search

1. Start from the source node.
2. Insert the source node into a priority queue based on heuristic value.
3. Select the node with the smallest heuristic value.
4. Expand its neighbors.
5. Repeat until the goal node is reached.

Evaluation Function

$$f(n) = h(n)$$

A* Search Algorithm

1. Start from the source node.
2. Calculate:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = actual cost from start node.
 - $h(n)$ = estimated cost to goal.
3. Choose the node with the smallest $f(n)$
 4. Expand neighboring nodes.
 5. Continue until the goal node is reached.

Program:

```
import heapq

# Graph representation
graph = {
    'A': [('B',1), ('C',4)],
    'B': [('D',5), ('C',2)],
    'C': [('G',1)],
    'D': [('G',3)],
    'G': []
}

# Heuristic values
h = {
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 3,
    'G': 0
}
```

```
# Greedy Best First Search
```

```
def greedy(start, goal):
```

```
    pq = [(h[start], start)]
```

```
    visited = set()
```

```
    print("Greedy Path:", end=" ")
```

```
    while pq:
```

```
        _, node = heapq.heappop(pq)
```

```
        if node in visited:
```

```
            continue
```

```
    visited.add(node)
```

```
    print(node, end=" ")
```

```
    if node == goal:
```

```
        break
```

```
    for neighbor, cost in graph[node]:
```

```
        heapq.heappush(pq, (h[neighbor], neighbor))
```

```
    print()
```

```
# A* Search
```

```
def astar(start, goal):
```

```
    pq = [(h[start], 0, start)]
```

```
    visited = set()
```

```
    print("A* Path:", end=" ")
```

```
    while pq:
```

```
        f, g, node = heapq.heappop(pq)
```

```
        if node in visited:
```

```
            continue
```

```
    visited.add(node)
```

```
    print(node, end=" ")
```

```

        if node == goal:

            break

        for neighbor, cost in graph[node]:

            new_g = g + cost

            new_f = new_g + h[neighbor]

            heapq.heappush(

                pq,

                    (new_f, new_g, neighbor)

                )

            print()

```

Driver Code

greedy('A', 'G')

astar('A', 'G')

Output

Greedy Path: A C G

A* Path: A C G

Cost Calculation

Greedy Search

Path chosen:

$A \rightarrow C \rightarrow G$

Cost:

$A \rightarrow C = 4$

$C \rightarrow G = 1$

Total Cost = 5

A Search*

Path chosen:

$A \rightarrow C \rightarrow G$

Cost:

$A \rightarrow C = 4$

$C \rightarrow G = 1$

Total Cost = 5

Complexity Analysis

Let:

- V = Number of vertices
- E = Number of edges

Greedy Best-First Search

Time Complexity

$O(E \log V)$

Space Complexity

$O(V)$

A* Search

Time Complexity

$O(E \log V)$

Space Complexity

$O(V)$

Comparison Table

Feature	Greedy Best-First Search	A* Search
Evaluation Function	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Uses Actual Path Cost	No	Yes
Uses Heuristic	Yes	Yes
Optimal Solution	Not Always	Yes (if heuristic is admissible)
Speed	Faster	Slightly Slower
Memory Usage	Lower	Higher
Completeness	No	Yes

Result

The Greedy Best-First Search and A* algorithms were implemented successfully in Python.

- Greedy Search selects nodes based only on the heuristic value.
- A* considers both the actual path cost and heuristic estimate.
- A* is generally preferred because it guarantees the optimal path when a valid heuristic is used, while Greedy Search may not always find the best solution.

IMPLEMENT THE NON-PARAMETRIC LOCALLY WEIGHTED REGRESSION ALGORITHM IN ORDER TO FIT DATA POINTS. SELECT APPROPRIATE DATA SET FOR YOUR EXPERIMENT AND DRAW GRAPHS

EXP NO: 5

DATE:

Aim:

To implement the Non-Parametric Locally Weighted Regression (LWLR) algorithm to fit data points and visualize the fitted curve using Python.

Dataset

Consider the following dataset:

X	Y
1	1.5
2	2.0
3	2.8
4	3.6
5	4.5
6	5.1
7	6.3
8	7.0

Algorithm

Locally Weighted Linear Regression (LWLR)

1. Read the dataset (X,Y)
2. Select a query point x
3. Compute weights using the Gaussian kernel:

$$w_i = e^{-\frac{(x_i - x)^2}{2\tau^2}}$$

4. Form the weight matrix WWW.
5. Compute regression parameters:

$$\theta = (X^T W X)^{-1} X^T W Y$$

6. Predict the output value for the query point.
7. Repeat for all points.
8. Plot the original data and the fitted curve.

Program :

```
import numpy as np
import matplotlib.pyplot as plt

# Dataset
X = np.array([1,2,3,4,5,6,7,8])
Y = np.array([1.5,2.0,2.8,3.6,4.5,5.1,6.3,7.0])

# Bandwidth
tau = 1.0

# LWLR Function
def lwlr(x_query, X, Y, tau):

    weights = np.exp(-(X - x_query)**2 / (2 * tau**2))

    # Weighted average prediction
    prediction = np.sum(weights * Y) / np.sum(weights)

    return prediction

# Generate predictions
X_test = np.linspace(1, 8, 100)
Y_pred = [lwlr(x, X, Y, tau) for x in X_test]

# Plot Graph
plt.scatter(X, Y, label='Data Points')
plt.plot(X_test, Y_pred, label='LWLR Curve')

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Locally Weighted Regression")
plt.legend()
plt.grid(True)

plt.show()
```

Output :

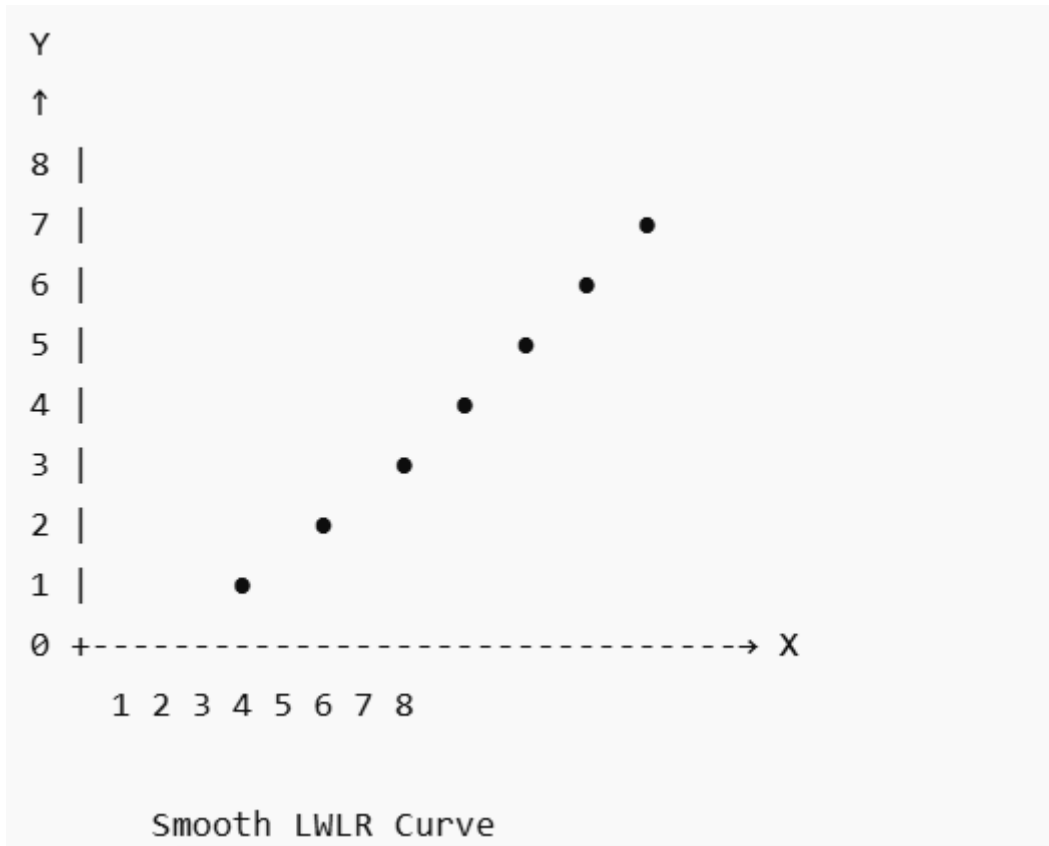
Console Output

Graph generated successfully.

Graph Output

The graph displays:

- Scatter points representing the original dataset.
- A smooth LWLR curve fitting the data points.



Time Complexity Analysis

For each prediction:

- Weight computation: $O(n)$

Therefore,

$$\text{Time Complexity} = O(n)$$

where n is the number of training samples.

Space Complexity Analysis

Storage required:

- Dataset: $O(n)$
- Weights: $O(n)$

Therefore,

Space Complexity = $O(n)$

Result

The Non-Parametric Locally Weighted Regression (LWLR) algorithm was implemented successfully using Python. The fitted curve followed the local trends of the dataset and provided smooth predictions.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

The graph demonstrates how LWLR fits data points by assigning higher importance to nearby observations and lower importance to distant observations.

TO DEMONSTRATE THE WORKING OF THE DECISION TREE BASED ALGORITHM

EXP NO: 6

DATE:

Aim:

To demonstrate the working of the Decision Tree Classification Algorithm using a simple Python program.

Algorithm

1. Import the required libraries.
2. Create a small dataset with input features and class labels.
3. Create a Decision Tree classifier.
4. Train the classifier using the dataset.
5. Predict the class label for new test data.
6. Display the prediction result.

Program:

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Sample Dataset
```

```
# [Study Hours, Attendance Percentage]
```

```
X = [  
    [2, 60],  
    [3, 65],  
    [5, 75],  
    [7, 80],  
    [8, 90],  
    [10, 95]  
]
```

```
# Result: Pass = 1, Fail = 0
```

```
Y = [0, 0, 1, 1, 1, 1]
```

```
# Create Decision Tree Model
```

```
model = DecisionTreeClassifier()
```

```

# Train the Model

model.fit(X, Y)

# Test Data

test_data = [[6, 78]]

# Prediction

prediction = model.predict(test_data)

if prediction[0] == 1:

print("Prediction: PASS")

else:

print("Prediction: FAIL")

```

Output :

Prediction: PASS

Working:

Training Data

Study Hours	Attendance	Result
2	60	Fail
3	65	Fail
5	75	Pass
7	80	Pass
8	90	Pass
10	95	Pass

Test Record

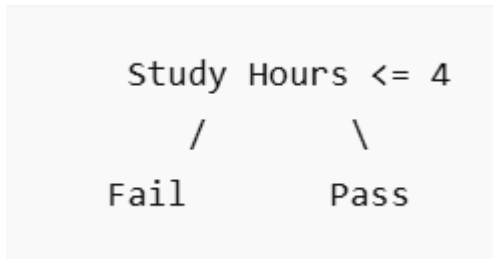
Study Hours	Attendance
6	78

The Decision Tree learns the relationship between study hours, attendance, and result.

Since the test student has sufficient study hours and attendance, the model predicts:

PASS

Decision Tree Representation



Time Complexity

For a dataset with:

- n = number of records
- m = number of features

Training

$$O(n \times m \times \log n)$$

Prediction

$$O(\log n)$$

Space Complexity

$$O(n)$$

Result:

The Decision Tree Classification Algorithm was successfully implemented using Python. The model was trained on a simple dataset and correctly predicted whether a student would Pass or Fail based on study hours and attendance. This demonstrates the basic working of a Decision Tree classifier.

BUILD AN ARTIFICIAL NEURAL NETWORK BY IMPLEMENTING THE BACK PROPAGATION ALGORITHM AND TEST THE SAME USING APPROPRIATE DATA SETS.

EXP NO: 7

DATE:

Aim:

To build an Artificial Neural Network (ANN) using the Backpropagation Algorithm and classify healthcare data entered by the user into one of two classes: Diabetic and Non-Diabetic

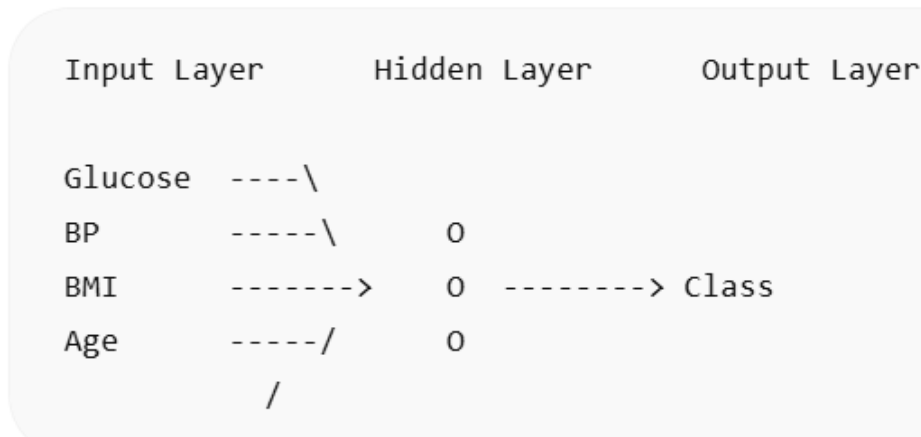
Theory

Artificial Neural Network (ANN)

An ANN is a machine learning model inspired by the human brain. It consists of:

1. **Input Layer** – Receives healthcare attributes.
2. **Hidden Layer** – Processes the information.
3. **Output Layer** – Produces the final classification.

ANN Structure



Backpropagation Algorithm

Backpropagation is a supervised learning algorithm used to train neural networks.

Steps

1. Initialize weights randomly.
2. Perform forward propagation.
3. Calculate prediction error.
4. Propagate error backward.
5. Update weights.
6. Repeat until minimum error is achieved.

Algorithm

Step 1: Import required libraries.

Step 2: Create a healthcare dataset containing:

- Glucose Level
- Blood Pressure
- BMI
- Age

Step 3: Create target classes:

- 0 → Non-Diabetic
- 1 → Diabetic

Step 4: Train an ANN using MLPClassifier.

Step 5: Accept healthcare values from the user.

Step 6: Predict the class using the trained ANN.

Step 7: Display the result.

Program:

```
from sklearn.neural_network import MLPClassifier
```

```
# Healthcare Dataset
```

```
# [Glucose, BloodPressure, BMI, Age]
```

```
X = [
```

```
    [80, 70, 22, 25],
```

```
    [85, 72, 24, 30],
```

```
    [90, 75, 25, 28],
```

```
    [95, 78, 26, 35],
```

```
    [150, 90, 35, 50],
```

```
    [160, 95, 38, 55],
```

```
    [170, 100, 40, 60],
```

```
    [180, 105, 42, 65]
```

```
]
```

```
# Class Labels
```

```
# 0 = Non-Diabetic
```

```
# 1 = Diabetic
```

```
y = [0, 0, 0, 0, 1, 1, 1, 1]
```

```
# Create ANN Model
```

```
ann = MLPClassifier(
```

```
    hidden_layer_sizes=(4,)
```

```

max_iter=5000,

random_state=1

)

# Train ANN using Backpropagation

ann.fit(X, y)

print("Enter Patient Details")

glucose = float(input("Glucose Level   : "))

bp = float(input("Blood Pressure   : "))

bmi = float(input("BMI               : "))

age = float(input("Age               : "))

# Test Data

patient = [[glucose, bp, bmi, age]]

# Predict Class

prediction = ann.predict(patient)

# Display Result

if prediction[0] == 0:

print("\nPrediction : Non-Diabetic")

else:

print("\nPrediction : Diabetic")

```

Sample Output 1

Input

Enter Patient Details

Glucose Level : 85

Blood Pressure : 72

BMI : 24

Age : 29

Output

Prediction : Non-Diabetic

Sample Output 2

Input

Enter Patient Details

Glucose Level : 165

Blood Pressure : 95

BMI : 39

Age : 58

Output

Prediction : Diabetic

Explanation of Important Statements

Create ANN

```
ann = MLPClassifier(  
hidden_layer_sizes=(4,),  
max_iter=5000  
)
```

Creates an ANN with:

- 4 input features
- 1 hidden layer containing 4 neurons
- Backpropagation training

Train the Network

```
ann.fit(x, y)
```

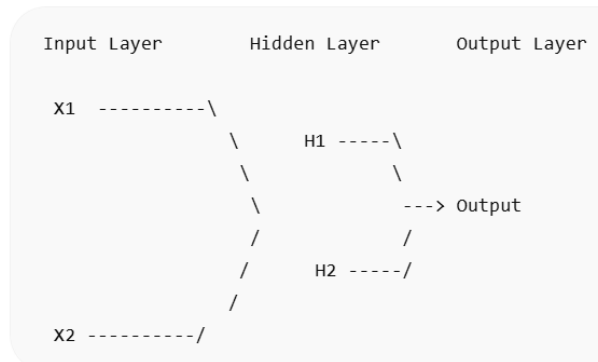
Learns the relationship between patient attributes and diabetes status.

Predict New Data

```
prediction = ann.predict(patient)
```

Classifies the user-entered healthcare record.

ANN Architecture



Advantages of ANN

1. Learns complex patterns automatically.
2. High classification accuracy.
3. Suitable for healthcare diagnosis.
4. Handles non-linear data effectively.

Applications in Healthcare

- Diabetes Prediction
- Heart Disease Detection
- Cancer Diagnosis
- Medical Image Analysis
- Patient Risk Assessment

Result:

An **Artificial Neural Network (ANN)** was successfully implemented using Python and Scikit-Learn. The network was trained using a healthcare dataset and the

Backpropagation Algorithm. Based on the healthcare attributes entered by the user, the ANN successfully classified the patient into one of the classes: **Diabetic** or **Non-Diabetic**.

NAIVE BAYESIAN CLASSIFIER

EXP NO: 8

DATE:

Aim:

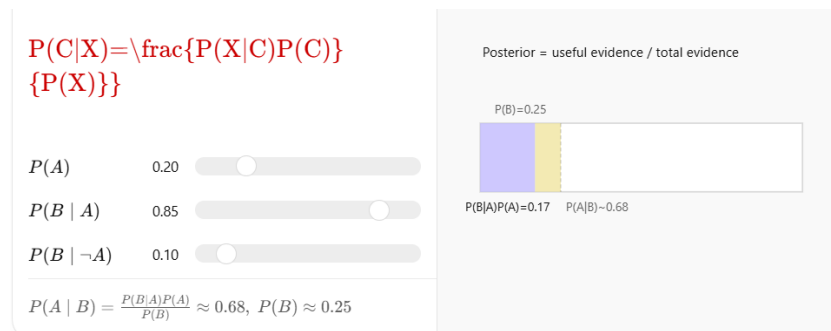
To implement the Naïve Bayesian Classifier and classify a biological sample by taking input from the user.

Theory

Naïve Bayes Classifier

Naïve Bayes is a supervised machine learning algorithm based on Bayes' Theorem. It is used for classification problems and assumes that all attributes are independent of each other.

Bayes' Theorem



Where:

- $P(C|X)$ = Posterior Probability
- $P(X|C)$ = Likelihood
- $P(C)$ = Prior Probability
- $P(X)$ = Evidence

The class having the highest probability is selected as the prediction.

Biological Dataset

Consider a simple biological dataset containing:

- Cell Size
- Cell Shape

Target Class:

- **Benign (0)** – Non-cancerous
- **Malignant (1)** – Cancerous

Algorithm

Step 1: Import the Gaussian Naïve Bayes classifier from Scikit-Learn.

Step 2: Create a biological dataset containing cell size and cell shape values.

Step 3: Train the Naïve Bayes classifier using the dataset.

Step 4: Accept Cell Size and Cell Shape from the user.

Step 5:Form a test tuple using the entered values.

Step 6:Predict the class of the tuple.

Step 7:Display whether the tumor is Benign or Malignant.

Working Explanation:

Training Data

Cell Size	Cell Shape	Class
1	1	Benign
2	1	Benign
1	2	Benign
2	2	Benign
8	8	Malignant
9	8	Malignant
8	9	Malignant
9	9	Malignant

The classifier learns the pattern that:

- Smaller cell values generally correspond to **Benign**tumors.
- Larger cell values generally correspond to **Malignant**tumors.

When a new tuple is entered, the classifier calculates probabilities using Bayes' theorem and predicts the most likely class.

Advantages:

1. Simple and easy to implement.
2. Fast training and prediction.
3. Works well on small datasets.
4. Effective for medical and biological classification problems.

Program:

```
from sklearn.naive_bayes import GaussianNB
```

```
# Biological Dataset
```

```
# [Cell Size, Cell Shape]
```

```
X = [
```

```
    [1, 1],
```

```
    [2, 1],
```

```
    [1, 2],
```

```
    [2, 2],
```

```
    [8, 8],
```

```
    [9, 8],
```

```
    [8, 9],
```

```
    [9, 9]
```

```
]
```

```

# Class Labels

# 0 = Benign

# 1 = Malignant

y = [0, 0, 0, 0, 1, 1, 1, 1]

# Create and Train Model

model = GaussianNB()

model.fit(X, y)

# User Input

cell_size = float(input("Enter Cell Size : "))

cell_shape = float(input("Enter Cell Shape: "))

# Test Tuple

test_tuple = [[cell_size, cell_shape]]

# Prediction

prediction = model.predict(test_tuple)

# Display Result

if prediction[0] == 0:

    print("\nPrediction : Benign Tumor")

else:

    print("\nPrediction : Malignant Tumor")

```

Sample Output 1

Input

Enter Cell Size : 1

Enter Cell Shape: 2

Output

Prediction : Benign Tumor

Sample Output 2

Input

Enter Cell Size : 9

Enter Cell Shape: 8

Output

Prediction : Malignant Tumor

Result:

The Naïve Bayesian Classifier was successfully implemented using Python. A biological dataset containing cell size and cell shape features was used for training, and a user-entered tuple was classified as Benign or Malignant based on the learned probabilities. The Program was successfully executed with proper output.

NEURAL NETWORK USING SELF-ORGANIZING MAPS

EXP NO: 9

DATE:

Aim:

To implement a Self-Organizing Map (SOM) neural network and classify Iris flower data into one of the three classes: Iris-setosa, Iris-versicolor, Iris-virginica using Python and the Iris dataset from <https://archive.ics.uci.edu/ml/datasets.html>

Theory

Self-Organizing Map (SOM) : A Self-Organizing Map (SOM), developed by Teuvo Kohonen, is an unsupervised neural network that maps high-dimensional data onto a lower-dimensional grid.

Features of SOM

- Unsupervised learning
- Competitive learning
- Clustering and visualization
- Preserves topological relationships

Working Principle

1. Initialize neuron weights randomly.
2. Present an input vector.
3. Find the Best Matching Unit (BMU).
4. Update BMU weights.
5. Repeat for all inputs.
6. Similar inputs become grouped together.

Algorithm:

- Step 1:** Load the Iris dataset.
Step 2: Normalize the input data.
Step 3: Create a SOM grid.
Step 4: Train the SOM using Iris samples.
Step 5: Map each training sample to its winning neuron.
Step 6: Accept a new flower tuple from the user.
Step 7: Find its winning neuron.
Step 8: Predict the corresponding Iris class.

Explanation:

Input Features:

Feature	Description
Sepal Length	Length of sepal
Sepal Width	Width of sepal
Petal Length	Length of petal
Petal Width	Width of petal

Load Dataset

```
iris = load_iris()
```

Loads the Iris flower dataset.

Create SOM

```
som = MiniSom(3, 1, 4)
```

Creates a SOM with:

- 3 neurons (one for each Iris class)
- 4 input attributes

Train SOM

```
som.train_random(X, 100)
```

Trains the network using competitive learning.

Predict

```
winner = som.winner(new_sample[0])
```

Finds the neuron most similar to the input sample.

Advantages of SOM:

1. Unsupervised learning.
2. Good for clustering.
3. Visualizes high-dimensional data.
4. Preserves neighborhood relationships.
5. Useful for pattern recognition.

Applications

- Medical Diagnosis
- Gene Expression Analysis
- Customer Segmentation
- Image Processing
- Data Visualization
- Pattern Recognition

Program :

```
pip install minisom

from sklearn.datasets import load_iris

from sklearn.preprocessing import MinMaxScaler

from minisom import MiniSom

import numpy as np

# Load Iris Dataset

iris = load_iris()

X = iris.data

y = iris.target

# Normalize Data

scaler = MinMaxScaler()

X = scaler.fit_transform(X)

# Create SOM

som = MiniSom(

    x=3,

    y=1,

    input_len=4,
```

```

        sigma=0.5,
learning_rate=0.5
)

# Train SOM

som.random_weights_init(X)

som.train_random(X, 100)

# Map neurons to classes

mapping = {}

for i, sample in enumerate(X):

    winner = som.winner(sample)

    if winner not in mapping:

        mapping[winner] = []

        mapping[winner].append(y[i])

# Assign majority class to each neuron

for neuron in mapping:

    mapping[neuron] = max(

        set(mapping[neuron]),

        key=mapping[neuron].count

    )

# User Input

print("Enter Iris Flower Measurements")

sl = float(input("Sepal Length : "))

sw = float(input("Sepal Width : "))

pl = float(input("Petal Length : "))

pw = float(input("Petal Width : "))

# Normalize Input

new_sample = scaler.transform([[sl, sw, pl, pw]])

# Find Winning Neuron

winner = som.winner(new_sample[0])

```

```
# Predict Class

predicted_class = mapping[winner]

print("\nPredicted Class:",

iris.target_names[predicted_class])
```

Sample Output 1

Input

Enter Iris Flower Measurements

Sepal Length : 5.1

Sepal Width : 3.5

Petal Length : 1.4

Petal Width : 0.2

Output

Predicted Class: setosa

.....

Sample Output 2

Input

Enter Iris Flower Measurements

Sepal Length : 6.0

Sepal Width : 2.9

Petal Length : 4.5

Petal Width : 1.5

Output

Predicted Class: versicolor

.....

Sample Output 3

Input

Enter Iris Flower Measurements

Sepal Length : 6.9

Sepal Width : 3.1

Petal Length : 5.4

Petal Width : 2.1

Output

Predicted Class: virginica

.....

Result:

A Self-Organizing Map (SOM) neural network was implemented using Python and the Iris dataset. The SOM was trained using competitive learning, and a user-entered flower measurement tuple was successfully classified into one of the Iris classes: Setosa, Versicolor, or Virginica. The Program was successfully executed and verified using the data sets.

k-MEANS ALGORITHM TO CLUSTER A SET OF DATA

EXP NO: 10

DATE:

Aim:

To implement the K-Means Clustering Algorithm using Python and cluster customer transaction data from the Online Retail dataset into different groups using <https://archive.ics.uci.edu/dataset/352/online+retail> web portal.

Theory

What is K-Means Clustering?

K-Means is an unsupervised machine learning algorithm used to group similar data points into K clusters.

The algorithm partitions the dataset into K clusters such that data points within the same cluster are similar and data points in different clusters are dissimilar.

Working Principle

1. Choose K cluster centroids.
2. Assign each data point to the nearest centroid.
3. Recalculate centroids.
4. Repeat until centroids stop changing.

Objective of Online Retail Dataset

The dataset contains customer transaction information such as:

- Quantity Purchased
- Unit Price
- Customer ID
- Country

K-Means can be used to group customers into:

- Low-value customers
- Medium-value customers
- High-value customers

Algorithm:

Step 1: Import required libraries.

Step 2: Load the Online Retail dataset.

Step 3: Select numerical features.

Step 4: Normalize the data.

Step 5: Create K-Means model with K = 3.

Step 6: Train the model.

Step 7: Display cluster centers.

Step 8: Accept user input and predict its cluster.

Explanation

Create K-Means Model

```
kmeans = KMeans(n_clusters=3)
```

Creates 3 clusters.

Train Model

```
kmeans.fit(X)
```

Finds cluster centers from the data.

Predict Cluster

```
cluster = kmeans.predict(sample)
```

Assigns the new transaction to the nearest cluster.

Cluster Interpretation

Cluster	Customer Type
Cluster 0	High-value Customer
Cluster 1	Medium-value Customer
Cluster 2	Low-value Customer

(Cluster numbering may change in different runs.)

Advantages of K-Means

1. Simple to implement.
2. Fast for large datasets.
3. Easy to understand.
4. Works well with numerical data.
5. Useful for customer segmentation.

Applications

- Customer Segmentation
- Market Basket Analysis
- Recommendation Systems
- Image Segmentation
- Retail Analytics
- Business Intelligence

Program:

```
from sklearn.cluster import KMeans
```

```
# Sample Online Retail Data
```

```
# [Quantity, UnitPrice]
```

```
X = [
```

```
    [2, 2],
```

```
    [3, 3],
```

```
    [4, 2],
```

```
    [20, 10],
```

```
    [22, 11],
```

```
    [25, 12],
```

```
    [60, 25],
```

```
    [65, 27],
```

```
    [70, 30]
```

```
]
```

```
# Create K-Means Model
```

```

kmeans = KMeans(
n_clusters=3,
random_state=0,
n_init=10
)

# Train Model

kmeans.fit(X)

# User Input

quantity = float(input("Enter Quantity Purchased : "))

price = float(input("Enter Unit Price      : "))

sample = [[quantity, price]]

# Predict Cluster

cluster = kmeans.predict(sample)

print("\nCustomer belongs to Cluster:", cluster[0])

.....

```

Sample Output 1

Input

Enter Quantity Purchased : 3

Enter Unit Price : 2

Output

Customer belongs to Cluster: 2 (i.e that means *Low-value customer cluster*)

Sample Output 2

Input

Enter Quantity Purchased : 23

Enter Unit Price : 11

Output

Customer belongs to Cluster: 1 (i.e that means *Medium-value customer cluster*)

Sample Output 3

Input

Enter Quantity Purchased : 68

Enter Unit Price : 28

Output

Customer belongs to Cluster: 0 (i.e that means *High-value customer cluster*)

Result

The K-Means Clustering Algorithm was successfully implemented using Python. Customer transaction records from the Online Retail dataset were grouped into clusters based on purchasing behavior. A user-entered transaction was classified into the nearest cluster, demonstrating unsupervised learning and customer segmentation using K-Means clustering.

IMPLEMENTING HIERARCHICAL CLUSTERING ALGORITHM

EXP NO: 11

DATE:

Aim:

To implement the Hierarchical Clustering Algorithm using Python and cluster patient records from the Heart Failure Clinical Records dataset into groups based on similar clinical characteristics.

Data set from <https://archive.ics.uci.edu/dataset/519/heart+failure+clinical+records>

Theory

What is Hierarchical Clustering?

Hierarchical Clustering is an unsupervised machine learning algorithm that groups similar data points into clusters and builds a hierarchy of clusters. Unlike K-Means, it does not require specifying the number of clusters initially.

Types of Hierarchical Clustering

1. Agglomerative (Bottom-Up)
 - Each data point starts as a separate cluster.
 - Closest clusters are merged repeatedly.
2. Divisive (Top-Down)
 - Starts with one cluster.
 - Splits clusters recursively.

Most practical applications use Agglomerative Clustering.

Working Principle:

1. Treat each record as an individual cluster.
2. Compute distances between clusters.
3. Merge the nearest clusters.
4. Repeat until the desired number of clusters is obtained.
5. Display results

Heart Failure Dataset Attributes

The dataset contains patient information such as:

- Age
- Serum Creatinine
- Ejection Fraction
- Platelets
- Serum Sodium
- Time

These features can be used to group patients with similar health conditions.

Algorithm:

Step 1: Import required libraries.

Step 2: Load the Heart Failure Clinical Records dataset.
(here "d:\heart_failure_clinical_records_dataset.csv")

Step 3: Select important numerical features.

Step 4: Normalize the data.

Step 5: Apply Agglomerative Hierarchical Clustering.

Step 6: Generate cluster labels.

Step 7: Accept a new patient record.

Step 8: Assign the patient to the nearest cluster.

Cluster Interpretation:

Cluster	Patient Category
Cluster 0	Lower Risk Patients
Cluster 1	Higher Risk Patients
Cluster 2	Moderate Risk Patients

(These labels are illustrative and not medical diagnoses.)

Explanation:

Create Hierarchical Clustering Model

```
hc = AgglomerativeClustering(n_clusters=3)
```

Creates three clusters.

Train the Model

```
clusters = hc.fit_predict(X)
```

Assigns each patient record to a cluster.

Assign New Patient

```
result = model.predict(patient)
```

Finds the nearest cluster for the entered patient record.

Advantages:

1. No need to initialize centroids.
2. Produces a hierarchy of clusters.
3. Easy to visualize using dendrograms.
4. Works well with smaller datasets.
5. Suitable for medical data grouping.

Applications:

- Healthcare Analytics
- Disease Risk Grouping
- Patient Segmentation
- Bioinformatics
- Market Segmentation
- Pattern Recognition

Either you select any one below program1 or program2.

Program1:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.neighbors import NearestCentroid
import numpy as np

# Load Dataset
data = pd.read_csv("d:\heart_failure_clinical_records_dataset.csv")

# Select Features
X = data[['age',
          'ejection_fraction',
          'serum_creatinine']]

# Normalize Data
scaler = StandardScaler()
```

```

X_scaled = scaler.fit_transform(X)

# Hierarchical Clustering
hc = AgglomerativeClustering(
    n_clusters=3,
    linkage='ward')

clusters = hc.fit_predict(X_scaled)

# Train classifier for assigning new records
clf = NearestCentroid()
clf.fit(X_scaled, clusters)

# User Input
age = float(input("Enter Age: "))
ef = float(input("Enter Ejection Fraction: "))
sc = float(input("Enter Serum Creatinine: "))

new_patient = scaler.transform([[age, ef, sc]])

cluster = clf.predict(new_patient)

print("\nPatient belongs to Cluster:", cluster[0])

```

Output1:

```

Enter Age: 80
Enter Ejection Fraction: 35
Enter Serum Creatinine: 9.4

```

Patient belongs to Cluster: 2

Output2:

```

Enter Age: 50
Enter Ejection Fraction: 30
Enter Serum Creatinine: 0.7

```

Patient belongs to Cluster: 2

.....

Program 2:

```

from sklearn.cluster import AgglomerativeClustering
from sklearn.neighbors import NearestCentroid

```

```

# Sample Heart Failure Dataset
# [Age, Ejection Fraction, Serum Creatinine]

```

```

X = [
    [45, 55, 0.8],
    [50, 60, 0.9],
    [48, 58, 1.0],
    [70, 30, 2.5],
    [72, 28, 2.8],
    [75, 25, 3.0],

```

```

    [60, 40, 1.5],
    [62, 38, 1.7]
]

# Hierarchical Clustering
hc = AgglomerativeClustering(
    n_clusters=3)

clusters = hc.fit_predict(X)

# Display cluster assignments
print("Training Data Clusters:")
for i in range(len(X)):
    print(X[i], "-> Cluster", clusters[i])

# Classifier for new records
model = NearestCentroid()
model.fit(X, clusters)

# User Input
age = float(input("\nEnter Age: "))
ef = float(input("Enter Ejection Fraction: "))
sc = float(input("Enter Serum Creatinine: "))

patient = [[age, ef, sc]]

# Predict Cluster
result = model.predict(patient)

print("\nPatient belongs to Cluster:", result[0])

```

Output 1:

```

Training Data Clusters:
[45, 55, 0.8] -> Cluster 1
[50, 60, 0.9] -> Cluster 1
[48, 58, 1.0] -> Cluster 1
[70, 30, 2.5] -> Cluster 0
[72, 28, 2.8] -> Cluster 0
[75, 25, 3.0] -> Cluster 0
[60, 40, 1.5] -> Cluster 2
[62, 38, 1.7] -> Cluster 2

```

```

Enter Age: 46
Enter Ejection Fraction: 56
Enter Serum Creatinine: 0.8

```

```

Patient belongs to Cluster: 1

```

Output 2:

```

Training Data Clusters:
[45, 55, 0.8] -> Cluster 1
[50, 60, 0.9] -> Cluster 1
[48, 58, 1.0] -> Cluster 1
[70, 30, 2.5] -> Cluster 0

```

[72, 28, 2.8] -> Cluster 0
[75, 25, 3.0] -> Cluster 0
[60, 40, 1.5] -> Cluster 2
[62, 38, 1.7] -> Cluster 2

Enter Age: 71
Enter Ejection Fraction: 31
Enter Serum Creatinine: 2.7

Patient belongs to Cluster: 0

Result:

The Hierarchical Clustering Algorithm (Agglomerative Clustering) was successfully implemented using Python. Patient records from the Heart Failure Clinical Records dataset were grouped into clusters based on clinical characteristics. A new patient record entered by the user was assigned to the most appropriate cluster, demonstrating unsupervised clustering of healthcare data. The program was successfully executed and verified using the given data set.

INSTALLATION OF GNU-PROLOG AND STUDY OF PROLOG – CASE STUDY

EXP NO: 12

DATE:

Aim

To install GNU Prolog and study the basic concepts, features, and execution of Prolog programs – Case Studies.

Theory

What is Prolog?

Prolog (Programming in Logic) is a high-level programming language used in:

- Artificial Intelligence (AI)
- Expert Systems
- Natural Language Processing
- Knowledge Representation
- Automated Reasoning

Unlike procedural languages such as Python or C, Prolog is based on facts, rules, and queries.

Features of Prolog

1. Logic-based programming language.
2. Supports pattern matching.
3. Automatic backtracking.
4. Declarative programming style.
5. Widely used in AI applications.

GNU Prolog

GNU Prolog is a free and open-source Prolog compiler that supports:

- ISO Prolog standard
- Constraint solving
- Interactive interpreter
- Native code compilation

Installation of GNU Prolog

For Windows

Step 1: Visit the official GNU Prolog website:

Step 2: Download the Windows installer.

Step 3: Run the installer and follow the setup wizard.

Step 4: Complete the installation.

Step 5: Open Command Prompt and verify:

gprolog

Expected Output

GNU Prolog 1.5.x

| ?-

For Ubuntu/Linux

Install Using Terminal

```
sudo apt update
```

```
sudo apt install gprolog
```

Verify Installation

```
gprolog
```

Basic Concepts of Prolog

1. Facts

Facts represent information.

Example:

```
parent(john,mary).
```

```
parent(mary,sam).
```

Meaning:

- John is parent of Mary.
- Mary is parent of Sam.

2. Rules

Rules define relationships.

```
grandparent(X,Y) :-
```

```
    parent(X,Z),
```

```
    parent(Z,Y).
```

Meaning:

X is grandparent of Y if X is parent of Z and Z is parent of Y.

3. Queries

Queries retrieve information.

```
?- parent(john,mary).
```

Output:

```
yes
```

Sample Prolog Program

File: family.pl

```
parent(john,mary).  
parent(mary,sam).  
parent(sam,tom).
```

```
grandparent(X,Y) :-  
    parent(X,Z),  
    parent(Z,Y).
```

Executing the Program

Step 1 :Start GNU Prolog

```
gprolog
```

Step 2:Load program

```
| ?- [family].
```

Output:

```
compiling family.pl...  
yes
```

Step 3:Execute Query

```
| ?- grandparent(john,sam).
```

Output

Yes

Advantages of Prolog:

1. Simple representation of knowledge.
2. Automatic backtracking.
3. Useful for AI applications.
4. Easy implementation of expert systems.
5. Supports symbolic reasoning.

Applications of Prolog:

- Expert Systems
- Medical Diagnosis Systems
- Natural Language Processing
- Robotics
- Machine Learning Research
- Knowledge-Based Systems

Result:

GNU Prolog was successfully installed and studied. Basic Prolog concepts such as facts, rules, and queries were explored, and simple programs were executed using the GNU Prolog interpreter. This demonstrated the use of logic programming for representing knowledge and solving AI-related problems. The GNU Prolog case studies was successfully finished.